```
{
     register int x=123;
     printf("\n Address of x = %u\n",&x);
}
```

Example 7

In C and C++, static variable initialisation is different. In C, static variables can only be initialised with constant values whereas in C++, this is not necessary. The program *Example 8* will not compile as a C program but it is valid when compiled with g++ as a C++ program.

```
#include<stdio.h>

int main()
{
     int x=123;
     static int y=x;
     printf("\ny = %d\n",y);
}
```

Example 8

The way case labels are treated in C and C++ is different. C++ allows constant integer variables as case labels, whereas C will give you an error. The program *Example 9* will give an error as a C program but as a C++ program, it compiles without any errors.

```
#include<stdio.h>

int main()
{
    int i=1;
    const int j=1;
    switch(i)
    {
        case j:
            printf("\nWorks for C++\n");
            break;
        case 2:
            printf("\nError for C\n");
            break;
        default:
            printf("\nCheck by compiling\n");
    }
}
```

Example 9

In C, uninitialised constant variables are just fine but these will lead to an error in the case of C++. For example, a program with the line of code *const int a;* will compile fine as a C program, whereas it will show a compile time error when compiled as C++ and display the error message *uninitialized const*. Another difference arises when the keyword *typedef* is used in C and C++. C++ treats the names of struct, union, enum, etc, as implicit *typedef* names. This is not the case with C, so program *Example 10* is a valid C program, whereas if compiled as a C++ program, you will get the error message *using typedef-name 'type' after 'struct'*.

```
#include<stdio.h>

typedef int type;
struct type
{
    type a;
};
struct type sv;

int main()
{
}
```

Example 10

In C, a pointer of type *void* is automatically cast into the *target* pointer type when assigned to a pointer of any other type. So, there is no need for explicit type-casting in C. Whereas in C++, explicit type-casting is required when a *void* pointer is assigned to a pointer of another type. So, the line *int *ptr = malloc(sizeof(int));* is fine in C, whereas in C++, you will get the compile time error message: *invalid conversion from 'void*' to 'int*'*. You can download this and all the other programs discussed in this article from *opensourceforu.com/article_source_code/june17cPlus.zip*. To save some space, I haven't used any code other than the feature being discussed in many of the programs. But you can add any valid C or C++ code in all these programs.

K & R style function definitions are still valid in C. But this will lead to an error in case of C++. So, the program *Example 11* is a valid C program but it is not a valid C++ program.

```
#include<stdio.h>
void fun(a, b)
int a;
int b;
{
     printf("\na = %d \nb = %d\n",a,b);
}
int main()
{
    fun(11,22);
}
```

Example 11